Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I plan on using loops, conditionals, lists, classes, and functions, improving my skills in function decomposition and function/variable naming.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I expect the logic to be easy, while the Drafter integration will be difficult. Drafter syntax is much more difficult than anything else we have done in this course and even with help from AI, I expect it to be difficult.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

I should first implement my functions, then get the code working in Thonny, then use AI to implement my code into Drafter.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

I don’t anticipate any distractions, as I plan to work in class or in the Trabant study lounge, which is generally quiet and a good work environment (hence why I go there).

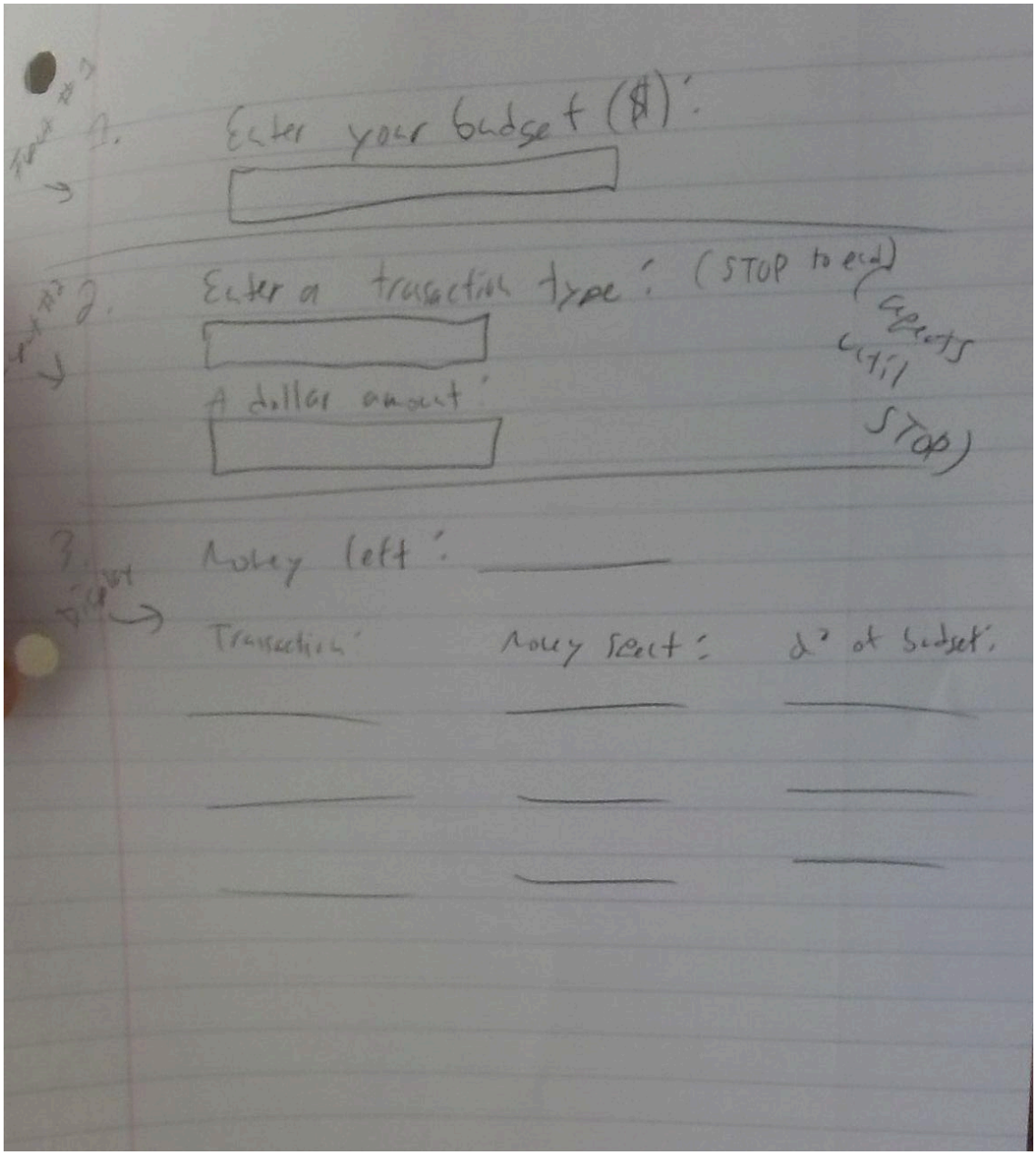
1) Site Diagram

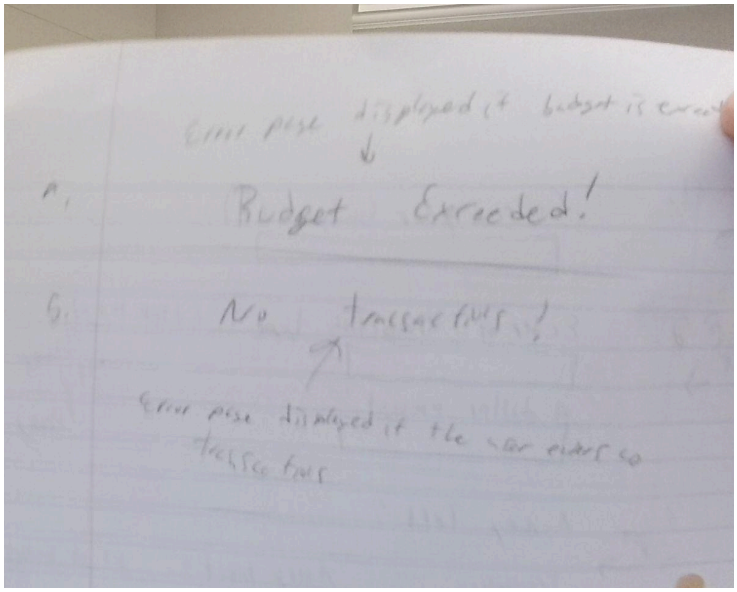
Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.


1. Enter your budget ($):
2. Enter a transaction type: (STOP to end)
A dollar amount:
3. Money left:
Transaction
Money spent:
$ of budget:



2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**My site will track a user's transactions. It will then display what percent of the budget the transaction takes as well as leftover money.**
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be	State: - Budget: int - The user's budget, - Transactions: list[Transaction] - The user's entered transactions - Names: list[str] - A list of all the names of the user's transactions - Money_amounts: list[float] - A list of all the money amounts of the

meaningfully modified throughout the site.	user's transactions - Percents: list[float] - A list of all the percents of the budget of the user's transactions
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	Transaction: - Type: str - The name of the transaction (user input) - Money: float - The amount of money spent on the transaction (user input) - Percent: float - The percent of the budget the transaction uses (calculated)
Routes (minimum 7) The names of your route functions	-Get_budget -Get_transactions -Get_transaction_amount -Get_transaction_type -Calculate_percents -Display_transactions -Too_much_spent -No_transactions
Pages (minimum 5) A brief description of the pages in your site	-Budget page: asks the user for budget -Transaction page: asks the user for transactions, goes until STOP is entered in the name -Display page: displays the budget and transactions, with percents for each transaction
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes	-TextBox: budget -TextBox: transaction name TextBox: transaction amount

For more information on these input types, see Drafter Documentation.	
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	-If statement that checks if “STOP” is entered to break first while loop -If statement checks if transaction list is empty, returns no transactions page -If statement checks if budget is exceeded, returns budget exceeded page
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	-While loop for adding transactions -for loop for turning what the user inputs into the transaction list -for loop for calculating percents

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

what is a state in drafter

In Drafter, a state is the collection of information your web application remembers while it is running.

Think of state as your app's memory.

For example, imagine you're building a study tracker.

Your state might look like this:

```
from drafter import *




@route

def home(state):

    state.study_hours = 0

    state.subject = "Computer Science"

    return state
```

Here, the state stores:

- `study_hours`: How many hours the user has studied.
- `subject`: Which subject the user is studying.

Why is state important?

State allows your application to remember information between user interactions.

For example:

1. The user clicks an "Add 1 Hour" button.
2. Drafter increases `state.study_hours` by 1.
3. The page updates to display the new number.
4. The state remembers the updated value.

A simple analogy:

Imagine your application is a person:

- State = The person's memory.
- User input = Information the person receives.
- Functions = Things the person does.
- Page = What the person communicates back.

Important: Drafter state is not the same as permanent storage. State generally lasts while the application session is running. If you want information to persist after the app is closed or refreshed, you can use browser local storage.

The most important thing to remember: State stores the variables your application needs to remember as users interact with it.

I can also show you how `state` works with `@route` and buttons in a small Drafter example.

what is a route function in drafter

In Drafter, a route function is a Python function that tells your web application what to do when a user visits a particular page or interacts with something that leads to that page.

Think of a route as instructions for a specific page in your website.

1. A basic route function

```
from drafter import *









@route



def home(state):



    return "Welcome to my website!"
```

Here's what each part means:

- `@route`: Tells Drafter that this function is a route.
- `def home(state)`: Defines a function named `home` that receives the application's state.
- `return`: Tells Drafter what page or content to display.

2. Routes can display different pages

Imagine you're building a study tracker. You might have two routes:

```
@route


def home(state):


    return "Welcome to the Study Tracker!"
```

```
@route


def about(state):


    return "This app helps you track study hours."
```

Each route represents a different page that the user can visit.

3. Routes can use and update state

Routes can also access information stored in state.

For example:

```
@route

def home(state):

    state.study_hours = 5

    return f"You have studied {state.study_hours} hours!"
```

This route updates the state and displays the information.

4. How routes fit into Drafter

User visits a page or clicks a link

Drafter calls the appropriate route function

Route processes information and possibly updates state

Drafter displays the returned page

The important difference

Concept	Purpose
`state`	Stores information your app remembers
`@route`	Identifies a function as a web route
`return`	Determines the page or content returned by the route
`def`	Defines the Python function

Remember: A route function is essentially the instructions for what your website should do when a user navigates to a particular page.

A useful next concept to learn is how Drafter buttons connect to route functions to let users interact with your application.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.